An Agent-Run Knowledge Base for AI Agent Controls

M Walker 10 October 2026

I built a knowledge base of AI agent security controls that agents research, verify and keep current. It covers 151 platforms and tools, and an enterprise questionnaire answers a client's concerns from it with every citation checked against the knowledge base itself.

◇ At a glance

The atlas holds 6,644 researched findings across 44 controls and 151 platforms and tools, backed by 3,655 sources that are each tiered by who published them. An evaluator agent verified 20 cells in a showcase run, each with the stored page, its hash and a quote a pure function found on that page. A recheck reproduced all 78 stored passages. The whole system runs on my own machine through a local gateway, with no API keys, and 160 automated tests pass.

◇ What it demonstrates

- **Agent design.** Four agent roles with separate runs and narrow permissions, none of which checks its own work.
- **Knowledge engineering.** One controls matrix projected into an OKF v0.2 tree, with frontmatter, chunks and an eligibility rule for agents.
- **Provenance.** Every verified claim rests on a stored page, a hash and a quote that can be checked again without trusting the agent.
- **Evaluation.** A labelled test set with deliberately altered claims, run against the real judge before it was allowed to write.
- **Governance.** Facts run with no human gate, while advice for an enterprise goes through a pull request with named approver roles.
- **Cost engineering.** Change detection by section hash, so only changed pages go back to a model.

◇ Why I built it

Clients ask me the same questions about agents. How does agent-to-agent authorisation work, and how do we stop one agent acting with another agent's permissions? Which gateway do we need in front of models and MCP tools? Can prompts, outputs and memory stay in Australia? Who approved that action, and can we prove it?

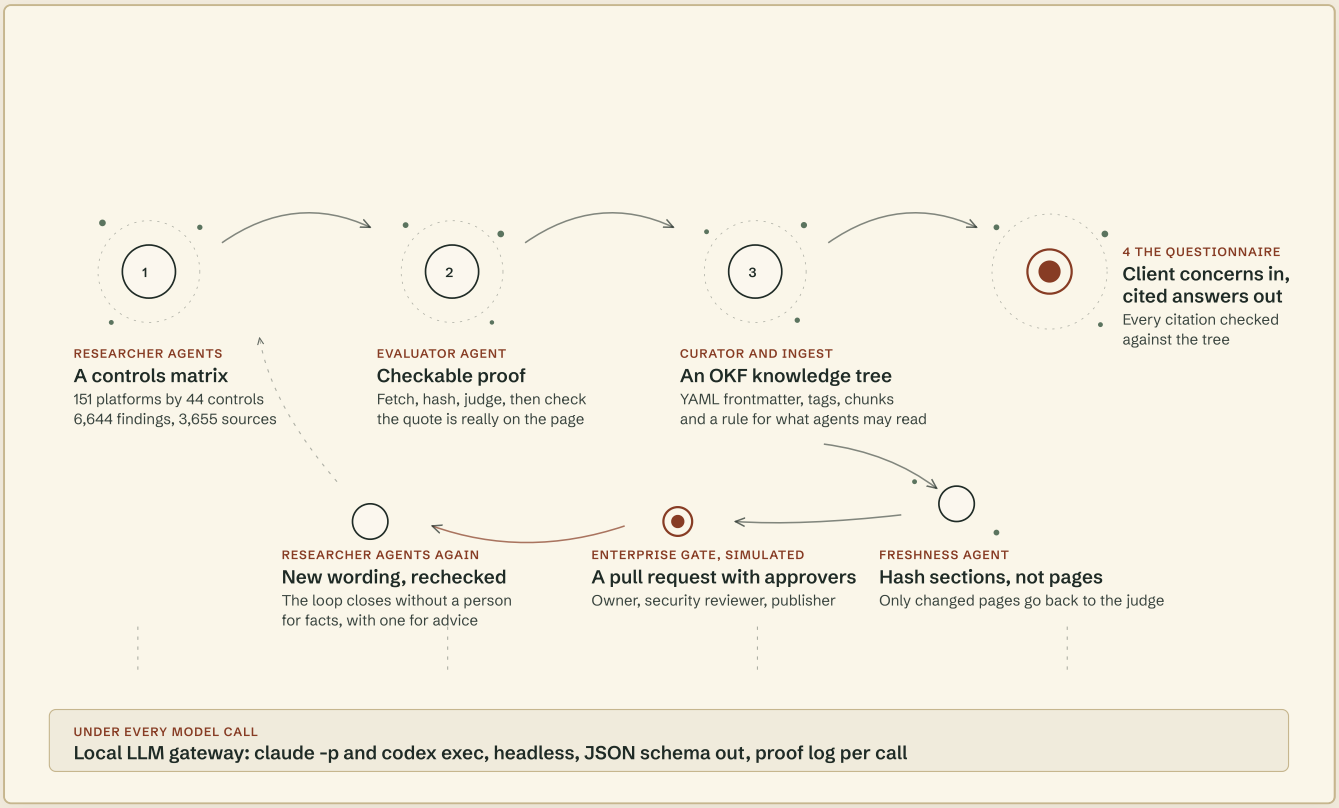
The answers are spread across hundreds of vendor documentation pages, and they change every few weeks. A model asked directly will answer fluently and often be out of date. I wanted a knowledge base that agents build and keep current themselves, where every fact traces to the exact passage it came from, and where it is clear what a person still has to decide.

The matrix

The atlas is one controls matrix. The 44 controls are rows, grouped into identity, authorisation, runtime, mediation and policy enforcement, observability, agentic MLSecOps and governance. Each platform or tool is a column, grouped by what it does: frontier AI providers, cloud platforms, coding agents, identity and secrets, AI security, AI governance, AI and MCP gateways, Australian sovereign hosting and 13 more.

Verdict	Cells	Meaning
Supported	725	The vendor's own documentation describes a native feature that meets the control
Partial	3,007	A native feature covers part of it, or needs configuration, add-ons or a higher tier
Not addressed	2,838	Checked, and the product does not offer it, usually because it sits outside its category
Unresolved	74	The documentation does not settle it either way

A verdict describes documented capability within the researched scope. It is not a deployment test, and the atlas says so wherever a verdict appears.



Four agents, each with one job

The roles come from a decision record, and each runs separately with no shared context.

Role	Does	Never does
Researcher	Reads vendor documentation and writes one finding per cell, with citations	Marks anything verified
Evaluator	Fetches each cited page, stores it by hash, has a model pick a verbatim passage, then checks the passage is really on the page	Judges a claim its own run drafted

Role	Does	Never does
Freshness	Hashes the sections a claim depends on and queues rechecks when they change	Rewrites a fact
Curator	Writes results through the store, runs the checks and commits with the run ID	Writes past a failed check

Every model call goes through a local gateway I built for this, which runs Claude Code and Codex headless on my own subscriptions and returns JSON that matches a schema. It is a separate project.

◇ Checkable proof

I did not want verification to mean an agent saying a claim was true. A verified cell in the atlas carries what anyone needs to check it again:

- the raw page, stored in a private evidence repository under its SHA-256
- the hash of the normalised text, the fetch time and the HTTP status
- a passage of 40 words or fewer, copied verbatim from that page
- the result of a pure-function check that the passage is an exact substring of the page text

The model decides whether the page supports the claim. It cannot invent the evidence, because a quote that is not on the page fails the check and the cell stays unverified. One command reopens every stored page and confirms every passage again, with no model involved.



The cell above is Mistral AI's credential brokerage. The judge found three passages across three of Mistral's own pages, one each for connector credentials, workload identity federation and API key rotation. The rejected cells are as useful. For Mistral's tool gateway the judge copied a whole permissions table, 41 words,

so the quote check failed. It also found that no page said API keys were scoped to connectors. The cell stayed unverified with both reasons recorded, ready for a researcher to split the claim.

◇ Testing the judge before trusting it

Before the evaluator could write anything it had to pass a labelled test set of 67 cases. Of those, 27 are deliberately altered claims: wrong features, wrong numbers, outdated statements, quotes that are not on the page, and a page carrying a prompt injection. The pass mark was no altered claim accepted and no more than one true claim in ten rejected.

Judge	Altered claims accepted	True claims rejected
Scripted replies	0 of 27	0 of 40
Haiku, after the fixture labels were corrected	0 of 22	1 of 28
Sonnet	0 of 22	2 of 28

Live judges see 22 of the altered cases, since five alter the scripted judge reply rather than the claim. Both models disagreed with the same five labels on their first runs, and each time the fixture was wrong, not the judge. A judge rigged to agree with everything shows what the hash and quote checks catch on their own: it would have let 20 of the 27 altered claims through. The deterministic checks stop invented quotes and weak sources. Wrong features and outdated claims still rest on the judge.

◇ The questionnaire

This is the part the rest of the atlas exists for. A client gives its maturity stage, what it runs and what worries it, in its own words. The example is a synthetic financial services firm piloting coding and support agents, running Claude Code on AWS with Okta, Wiz and Microsoft Purview.

1. **Route.** A small model maps each concern to the controls it is about, and to the tool groups where an answer might sit. A question about which gateway to use goes to the model and tool gateway controls and to the AI and MCP gateways group.
2. **Retrieve.** Records come from the OKF tree. Where nothing in the client's stack supports a control, the best-supported alternatives from the routed groups are added. Kong AI Gateway came up for agent-to-agent mediation.
3. **Answer.** A second model answers each concern from those records alone. Every claim must name a record and a source URL, and both are checked against the records it was given.
4. **Frame.** The client's maturity stage picks the engagement package and the offerings that fit it.

The run took five model calls, about 45,000 tokens in and 9,600 out. Of 38 citations, 37 passed the check. The one that failed, a claim about Okta's gateway that came back with no source URL, is marked as failed in the answer the client sees. The answers are direct about gaps. The records do not confirm that prompts, outputs and memory can stay in Australia end to end. Claude Code's auto memory has no documented retention limit. Wiz and Okta do not cover residency. Each answer ends with what the knowledge base does not settle, which is where an engagement starts.

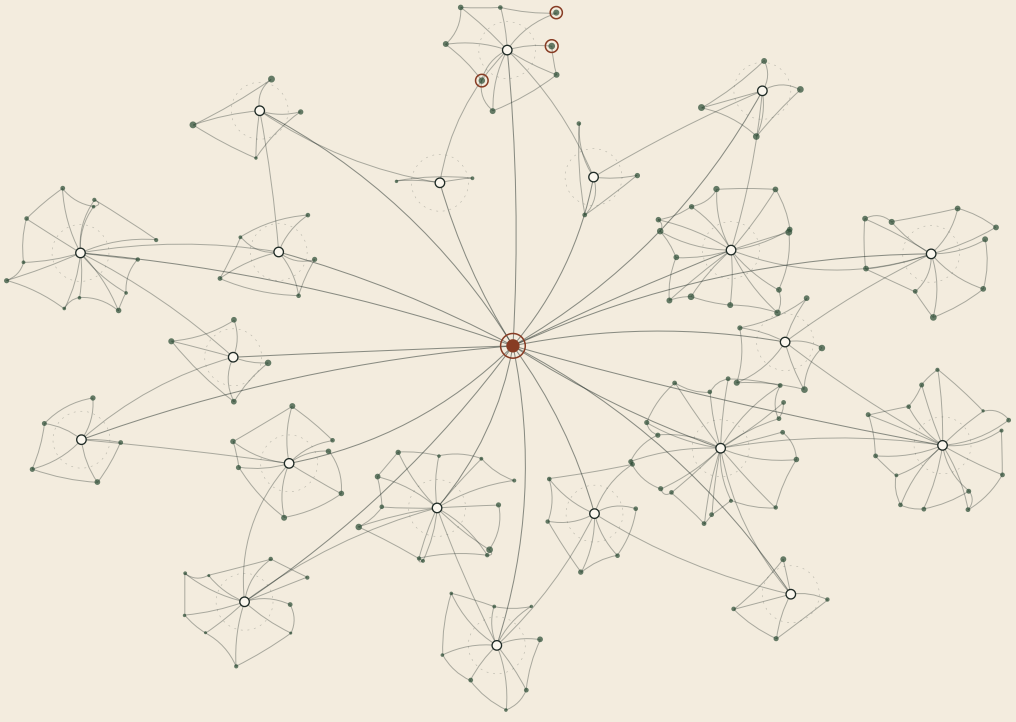
An OKF knowledge tree

The curator projects the atlas into an Open Knowledge Format v0.2 tree of 6,992 concepts, built to sit beside my business knowledge tree. Each matrix cell becomes a concept with YAML frontmatter and Markdown chunks:

```
---
type: "Platform capability"
title: "Anthropic: Human Approval"
classification: "C1"
verification: "unverified"
tags: ["frontier-ai", "authorisation", "supported"]
sources:
  - id: "an-permissions"
    resource: "https://platform.claude.com/docs/en/managed-agents/permission-policies"
    locator: "always_ask, auto and tool confirmation"
atlas:
  agent_eligible: true
  eligibility_basis: "owner-accepted"
---

<!-- chunk: answer -->
## Answer
...
```

Tags come from the column group, the control category and the verdict. Sources keep their locators and tiers. The eligibility rule decides what an agent may retrieve. In an enterprise tree only verified records would qualify. For this proof of concept I also accepted the researched cells myself, and those are labelled as owner accepted, never as verified. A cell the evaluator checked and could not verify drops out of the tree, because an evaluator verdict replaces my acceptance.



Keeping it fresh, and where a person comes back in

The freshness agent watches only the sections of a page that a claim depends on, so a footer edit raises nothing. In a scripted demonstration, the Amazon Bedrock model card for Claude Sonnet 5.5 gains an Australian Geo inference route. The agent detects the changed section and finds the two findings that depend on it. It marks them stale and queues evaluator jobs, with no model call at any step.

For facts in this atlas there is no human gate. The rules are strict enough that an unchecked claim is still labelled as one, and a wrong claim is caught by the next evaluator run. Enterprise knowledge is different. The same change becomes a pull request with the original wording, the evidence hash, the verbatim passage and proposed new wording, routed through a knowledge owner, a security reviewer and a publisher. The pull request also separates the fact from the judgement that stays with a person: whether client guidance that recommended other models for Australian processing should now include Sonnet 5.5. The approval chain is a labelled simulation with synthetic roles.

What it costs

The gateway logs tokens and a notional price for every call. Calls on my subscriptions have no marginal cost, so the price is the tool's own estimate at list rates.

Work	Model calls	Notional cost
One Sonnet judge call on a real cell	1	about USD 0.045
The 41-cell showcase run	41	about USD 1.85
The client questionnaire	5	about USD 0.30
Verifying every supported and partial cell once	about 4,567	about USD 206 with Sonnet

After the first pass, freshness sends only changed pages back to the judge, so keeping the atlas current costs a small share of building it.

Lessons and where the pattern goes next

- **Separate the proof from the judgement.** A hash and a substring check are cheap and certain. The model's job shrinks to the part that needs judgement.
- **Test the judge against the failures you fear.** The gullible-judge run showed exactly which failures the deterministic checks cover and which still need a model.
- **Label every state honestly.** Researched, owner accepted, verified and withheld are different things, and a reader should never have to guess which one they are looking at.
- **Keep advice with a person.** Agents can keep facts current. Whether a fact changes a recommendation is a decision with an owner.

The same pattern fits any knowledge an enterprise's agents act on: control libraries, supplier assurance, policy registers, product catalogues. Each needs the same parts: tiered sources, checkable proof, change detection and a clear line between facts and decisions.

◇ Limits

- Verdicts describe documented capability, not tested tenant behaviour.
- 20 cells carry agent proof. 652 high-risk cells wait on a second model family that is not set up yet.
- The research dates from 7 and 9 October 2026, and vendor products change.
- The client is synthetic, and no client or employer material is in the knowledge base.

◇ Sources

- Open Knowledge Format README, Google Cloud
- Workload identity and connector management, Mistral AI documentation, fetched 10 October 2026
- Claude Sonnet 5.5 model card, Amazon Bedrock User Guide

Figures come from the project's own repository, evaluator runs and gateway proof log as of 10 October 2026.